

XIAMEN UNIVERSITY

COM102 – Programming Language C

Tutorial

Dr. Najla'a A. M. Draib

Exercise

Employee Salary Validation (C)

Task 1: Define the Structure

Create a structure Employee with:

- `name` (string, max 30 characters)
- `empId` (integer)
- `salary[3]` (float – salary for 3 months)

Exercise 1

Task 2: Input & Validation

Write a function `inputEmployee()` to:

- Input the employee's name and ID
- Input **3 salary values**
- **Reject negative salary values** and ask the user to re-enter

Exercise 1

Task 3: Display Information

Write a function `displayEmployee()` to:

- Display the employee's **name, ID, and 3 valid salaries clearly**

```c
struct Employee {
    char name[30];
    int empId;
    float salary[3];
};

void displayEmployee(struct Employee e);

int main() {
    struct Employee emp;

    // TODO 1: Read employee name (use fgets)
    // Hint: remove the '\n' using strcspn

    // TODO 2: Read employee ID (use scanf)

    // TODO 3: Read 3 salaries using a loop
    // TODO 4: Validate: if salary is negative, keep asking until it is positive
    // Hint: use do...while OR while loop

    displayEmployee(emp);
    return 0;
}

void displayEmployee(struct Employee e) {
    // TODO 5: Print name and ID
    // TODO 6: Print the 3 salaries using a loop
}
```

```
/*
Description:
This program stores and displays employee information using a structure.
Input the employee name, ID, and three monthly salaries in main().

Each salary must be positive. Re-enter the salary if a negative value is
entered.

Complete the TODO sections below.
*/
```

Exercise 2

You are tasked with building a simple **name formatter tool** for a university's data-entry process. The tool processes one name at a time and applies operations based on user commands. Your program should repeatedly ask the user to enter student names (one by one) and offer formatting and analysis options.

Exercise 2

Task 1: Input and Process Names in a Loop

- Ask the user how many names they want to process (max 20).
- Use a loop to process each name one at a time.
- **For each name:**
  - Ask the user to input a name (use char name[50]).
  - If the input is empty, use `continue` to the next iteration.
  - If the user enters "`No`" (case-sensitive), terminate the loop early.
  - Otherwise:
    - Check if the name is longer than a given number N (you can define or ask N).
    - Replace all spaces in the name with hyphens.

Exercise 2

Task 2: Capitalize the First Letter of Each Word

Write a function to capitalize the first letter of each word in a name string. This is commonly used in name formatting for forms, certificates, and user interfaces.

Example:

- Input: "mohamed ali“
- Output: "Mohamed Ali“

Exercise 3

Task 1: Printing Array Elements Using Pointer Notation

Design and implement a C program that:

a. Takes an array of integers as input in the main() function.
b. Implements two separate functions:
   - One that prints the array using **pointer notation**.
   - Another that prints the array using **array notation**.

Exercise 3

Task 2: Array Printing with Special Modification

- Add a new function `modifyDivisibleByThree()` that:
  - Takes the array and its size
  - Doubles every element divisible by 3
  - Returns nothing (modifies array in-place)
  - Call this function before printing in `main()`

Exercise 3

Task 3: New Search Function Requirement

- Implement `searchNumber()` that:
  - Takes the array, its size, and a target number
  - Returns the first index where the number is found (or -1 if not found)
- Display search results in `main()`

Exercise 3

Task 4: Implement a function to count odd numbers in the modified array

- Add a function countOddNumbers() that:
- Takes the array and its size as input
- Returns the count of odd integers
- Print the result in main() after array modification